



Daver

A conversational SQL interface for fetching data using
natural language

Project for KV Engineering of AI-intensive System

Matthias Böhm, K11907932

Lukas Kurz, K12007739

Table of Contents

Goal of the system	2
Requirements	2
Functional Requirements	2
Non-Functional Requirements	3
Use Case Descriptions	4
Use Case 01 (UC01)	4
SUC01-1	5
SUC01-2	6
Use Case 02 (UC02)	7
SUC02-1	8
Use Case 03 (UC03)	9
SUC03-1	10
Use Case 04 (UC04)	11
SUC04-1	12
Use Case 05 (UC05)	13
SUC05-1	14
Use Case Diagram	15
Implemented Use Cases	16
Traceability Matrix	16
Domain Model	17
Architecture Diagram	18
Components Description	18
Non-AI Components:	18
AI Components:	19
Design Questions	20

Goal of the system

The main goal of the daver system is to make data stored in company databases and data warehouses easily accessible and understandable for non-technical users. By enabling the translation of natural language queries into system-specific or domain-specific query languages, the system allows users to extract meaningful insights, perform statistical analysis, and access relevant information without requiring knowledge of programming or database query languages. This approach aims to simplify and accelerate the data analysis process across all levels of an organization.

Requirements

Functional Requirements

Req01: The daver system shall translate natural language queries into domain specific database queries.

Req02: When a user submits a natural language query, the daver system shall return relevant data from the database.

Req03: The daver system shall display the generated query for debugging and understanding.

Req04: When a user wants to display data visually, daver shall generate some charts from the queried data.

Req05: When a query returns unsatisfactory results to a user, daver shall accept negative feedback from the user.

Req06: If the database schema changes, the daver system shall adapt to the new database schema.

Req07: The daver shall be able to read a table restriction and access restriction file.

Req08: The daver system shall detect and prevent attempts to perform unauthorized operations.

Req09: When a user adds additional information to a natural language query, daver shall adapt its query to the new information.

Req10: When a query returns unsatisfactory results, daver shall redo the query to produce a different result.

Req11: The daver system shall keep a record of executed queries by users that can be referenced or reused.

Non-Functional Requirements

NfReq01 [Performance]: The daver system shall translate natural language queries to SQL and return results within 10 seconds for queries of moderate complexity.

NfReq02 [External Interfaces]: The daver system shall provide connectors to different database systems, such as MongoDB, Postgres, MySQL.

NfReq03 [Usability]: The daver system shall provide responses that non-technical users can understand without requiring knowledge of database terminology.

NfReq04 [Constraints - Security]: The daver system shall enforce database access permissions consistent with existing user roles and privileges.

NfReq05 [Performance]: The daver system shall not unnecessarily hold up database connections.

NfReq06 [Attributes - Reliability]: The daver system shall achieve at least 70% accuracy in natural language query translation to SQL.

NfReq07 [Attributes - Maintainability]: The daver system shall store logs for queries based on user feedback.

NfReq08 [Privacy and Security]: The daver system shall not store any data queried from the database.

NfReq09 [Attributes - Adaptability]: The daver system shall support the english and german languages.

NfReq11 [Constraints - Privacy]: The daver system shall not write any data in the database.

Use Case Descriptions

Use Case 01 (UC01)

ID	UC01	
Description	Have the system translate a natural language input query to a domain specific SQL query	
Actors	End-User, Machine Learning Model	
Stakeholders:	End-User, Database Administrators, Software Developers	
Pre-Conditions	User is allowed to access the database. System is connected to a database and has the schema information	
Success end condition:	Valid SQL query is generated	
Failure end condition:	Output SQL query is invalid. Natural language cannot be parsed	
Main Success Scenario		Linked UCs
1	The user enters a natural language query into the systems input interface	
2	The system receives the query and and initiates processing it to feed it to the model	SUC01-1
3	The AI model translates the query into a SQL command that matches the schema	SUC01-2
4	The system validates the generated SQL against the database schema	
5	The resulting query is sent to the database to be executed	
6		
Alternative Scenarios		
3.A1	The systems detects ambiguity in the natural language query	
3.A2	The system requests clarification from the user	
Exception Scenario		
4.A1	The database schema is not available	
4.A2	The system throws an error and prompts the user to define the schema	

SUC01-1

Supporting Use case: Validate and process user input		
ID	SUC01-1	
Description	Process and validate natural language input query before passing it to the ML model	
Actors	End-user	
Stakeholders:	End-user, Software Developers	
Pre-Conditions	A natural language query has been entered and been sent to the system	
Success end condition:	The natural language query is validated and ready for model translation	
Failure end condition:	The input query produces an error during processing	
<u>Main Success Scenario</u>		
1	The system receives the natural language query	
2	The system performs pre processing and validation steps (trimming, checking for non-empty input, ...)	
3	The system confirms that the query is valid and sends it to the ML model for translation	
<u>Alternative Scenarios</u>		
<u>Exception Scenario</u>		
2.A1	The system throws an error during processing	
2.A2	The error is sent to the user and asks for adapting the query	

SUC01-2

Supporting Use case: Log Query Translation Request		
ID	SUC01-2	
Description	Record each translation request	
Actors	System	
Stakeholders:	Database Administrator, System Developers	
Pre-Conditions	A natural language query has been submitted for translation	
Success end condition:	The query details are logged and recorded	
Failure end condition:	No data is being logged and an error is thrown	
<u>Main Success Scenario</u>		
1	The system passes the natural language query	
2	The system records the natural language query	
3	The system passes the natural language query to the model	
4	The system records the output query of the model	
5	The system records all the given metadata associated with the translation request	
6	The system confirms that the data has been logged	
<u>Alternative Scenarios</u>		
5.A1	There is no metadata available so there is nothing stored	
<u>Exception Scenario</u>		
2,4,5.A1	The system throws an error during logging	
2,4,5.A2	The system still continues with the request despite the logging error	

Use Case 02 (UC02)

Use case: Execute Query and display result to user		
ID	UC02	
Description	Execute a valid query on the database and return the result to the user	
Actors	End-User	
Stakeholders:	End-User, Database Administrators, Software Developers	
Pre-Conditions	User is allowed to use the system. Valid SQL query has been generated. System is connected to the database	
Success end condition:	SQL query is successfully executed. Resulting data is displayed to the user	
Failure end condition:	Query execution fails. Data can not be displayed to user	
Main Success Scenario		Linked UCs
1	The system receives a valid SQL query	Result from UC01
2	The system forwards the query to the database to be executed	
3	The database executes the query and returns the result	
3	The resulting data is sent to the user	
4	The data is formatted and presented to the user	SUC02-1
6		
Alternative Scenarios		
4.A1	The data is not simply to be formatted but also to be worked with (visualization)	
4.A2	The data is forwarded to an appropriate system that manages further processing	
Exception Scenario		
2.A1	A connection to the database cannot be established	
2.A2	The user gets an error that the database connection is not working	
2.A3	The user is prompted to update the database configuration	
3.A1	The query execution fails	
3.A2	The query gets sent to the user	
3.A3	The user is asked to validate the query	

SUC02-1

Supporting Use case: Format and present query results		
ID	SUC02-1	
Description	Format the result of the database execution and present it to the user	
Actors	End-user, System	
Stakeholders:	End-user, Software Developers	
Pre-Conditions	A query result has been sent to the user	
Success end condition:	The data is displayed to the user in a readable way	
Failure end condition:	The data visualization fails	
Main Success Scenario		
1	The query result is sent to the user	
2	The system receives the data and formats it	
3	The formatted data is presented to the user	
Alternative Scenarios		
2.A1	The data is not simply to be formatted but also to be worked with (visualization)	
2.A2	The data is forwarded to an appropriate system that manages further processing	
Exception Scenario		
2.A1	The system throws an error during formatting	

Use Case 03 (UC03)

Use case: Display debug information		
ID	UC03	
Description	The system provides transparency by displaying the generated SQL query and other information	
Actors	End-User	
Stakeholders:	End-User, Database Administrators, Software Developers	
Pre-Conditions	SQL query has been successfully generated by the ML model. Debugging and logging is enabled	
Success end condition:	The generated Query and additional information is displayed to the user	
Failure end condition:	Debug information is unavailable due to internal system errors	
<u>Main Success Scenario</u>		Linked UCs
1	Debugging is enabled	
2	Debug information is being recorded by the system	
3	Debug information and query are sent to user alongside the result	SUC03-1
4	Query and debug information is displayed to the user	
<u>Alternative Scenarios</u>		
1.A1	Debugging is disabled	
1.A2	No additional information is recorded or displayed to the user	
<u>Exception Scenario</u>		
2.A1	System fails to record debugging information	
2.A2	Error is being shown that debugging failed	

SUC03-1

Supporting Use case: Retrieve Debug Logs		
ID	SUC03-1	
Description	Fetch and display the recorded debug logs	
Actors	End-user, Software Developers	
Stakeholders:	End-user, Software Developers, Database Administrator	
Pre-Conditions	Debug logging must be active and logs must be stored during processing	
Success end condition:	The relevant logs are retrieved and presented to the user	
Failure end condition:	Logs are not accessible	
Main Success Scenario		
1	A result is sent to the user while the debug mode is active	
2	The debug logs (natural language query, translated SQL query, metadata, ...) is queried from where it is stored	
3	The queried debug logs are sent to the user alongside the result	
4	The debug logs are displayed to the user	
Alternative Scenarios		
2.A1	No debug logs are available	
2.A2	An information that no logs are available is sent to the user	
Exception Scenario		
2.A1	The retrieval of the logs fail	
2.A2	An error message is sent to the user	
4.A1	Displaying the logs fails	

Use Case 04 (UC04)

Use case: Handle user feedback		
ID	UC04	
Description	Allow the end-user to provide feedback on the result and let them re-execute the query	
Actors	End-User, Machine learning model	
Stakeholders:	End-User, Database Administrators, Software Developers	
Pre-Conditions	The system has executed a query and displayed it to the user. A feedback mechanism is available	
Success end condition:	User feedback is recorded. System processed feedback and returns alternative result	
Failure end condition:	Feedback is not recorded or query re-execution is not working	
<u>Main Success Scenario</u>		Linked UCs
1	User reviews the result and is unsatisfied with it	
2	User clicks a "not satisfied" button	
3	The system records the previous query along with the feedback	SUC04-1
4	The system triggers a query redo mechanism, incorporating some random noise or alternative logic	
5	The resulting query is executed and displayed to the user	
<u>Alternative Scenarios</u>		
2.A1	The user can provide detailed feedback like suggestions	
<u>Exception Scenario</u>		
3.A1	The system is unable to store the previous query and the feedback	
4.A1	The system is unable to process the input together with the added feedback or noise	

SUC04-1

Supporting Use case: Record user feedback		
ID	SUC04-1	
Description	The user feedback is recorded and stored	
Actors	End-user, System	
Stakeholders:	End-user, Software Developers	
Pre-Conditions	A query must have been executed and the result presented to the user. The user must have given feedback	
Success end condition:	User feedback is recorded and stored alongside with context details	
Failure end condition:	Storing the feedback fails	
Main Success Scenario		
1	The user marks the query result as unsatisfactory	
2	The system records and stores the feedback	
3	The system also records other data like the natural language query, executed query and metadata	
4	The logs are stored until needed for further analysis or debugging	
Alternative Scenarios		
1.A1	The user gives more detailed feedback or suggestions	
1.A2	The detailed feedback is stored	
Exception Scenario		
2.A1	The system is unable to store the feedback	
2.A2	The system continues without storing the feedback	
3.A1	The system is unable to store the other data	
3.A2	The system continues without storing the other data	

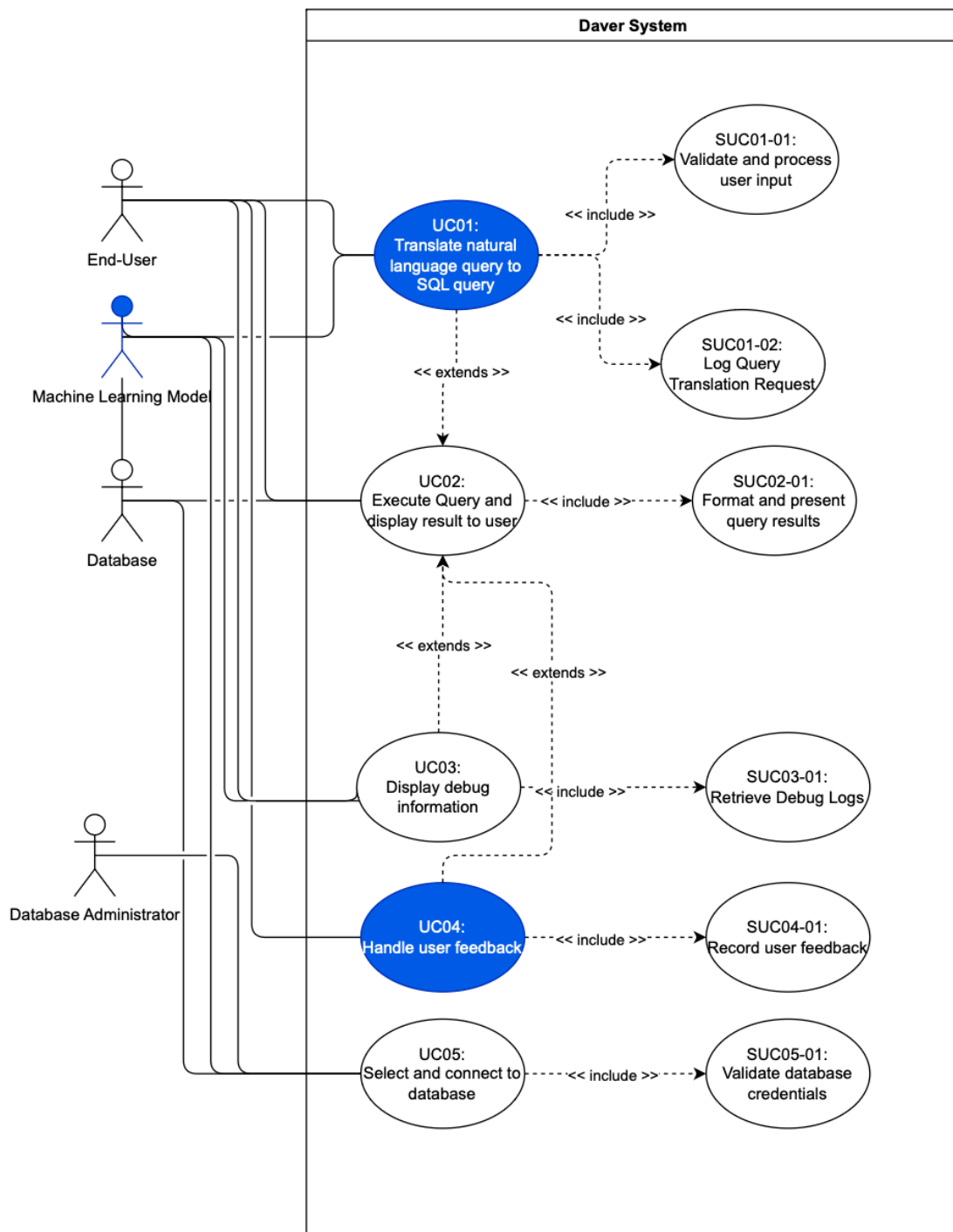
Use Case 05 (UC05)

Use case: Select and connect to database		
ID	UC05	
Description	The user is able to configure the desired database and provide schema information	
Actors	End-User, Database Administrators	
Stakeholders:	End-User, Database Administrators	
Pre-Conditions	The system is running. The user has a database running	
Success end condition:	The database connection is successful	
Failure end condition:	The connection attempt fails	
Main Success Scenario		Linked UCs
1	The user is provided with input fields to configure the database connection	
2	The configuration data is sent to the system	
3	The system initiates the connection	SUC05-1
4	The connection to the database is activated	
5	The user gets the information that the connection was successful	
Alternative Scenarios		
5.A1	The user gets asked if the credentials should be stored for future sessions	
Exception Scenario		
3.A1	The connection is not successful	
3.A2	The user gets the information that the connection could not be established	
3.A3	The user can re-enter the configuration	

SUC05-1

Supporting Use case: Validate database credentials		
ID	SUC05-1	
Description	The credentials to connect to the database are validated	
Actors	End-user	
Stakeholders:	End-user, Database Administrator	
Pre-Conditions	The user has entered credentials for the database connection	
Success end condition:	Credentials are verified and user is able to gain access to the database	
Failure end condition:	The credentials are rejected by the database	
Main Success Scenario		
1	The system receives the user credentials	
2	The authentication request with the credentials is sent to the database	
3	The database validates the credentials	
4	The database confirms validation	
5	The System proceeds with connecting the user with the database	
Alternative Scenarios		
Exception Scenario		
3.A1	Database credentials are invalid	
3.A2	Error message is sent to the user that credentials are invalid	
3.A3	User is prompted to enter credentials again	

Use Case Diagram



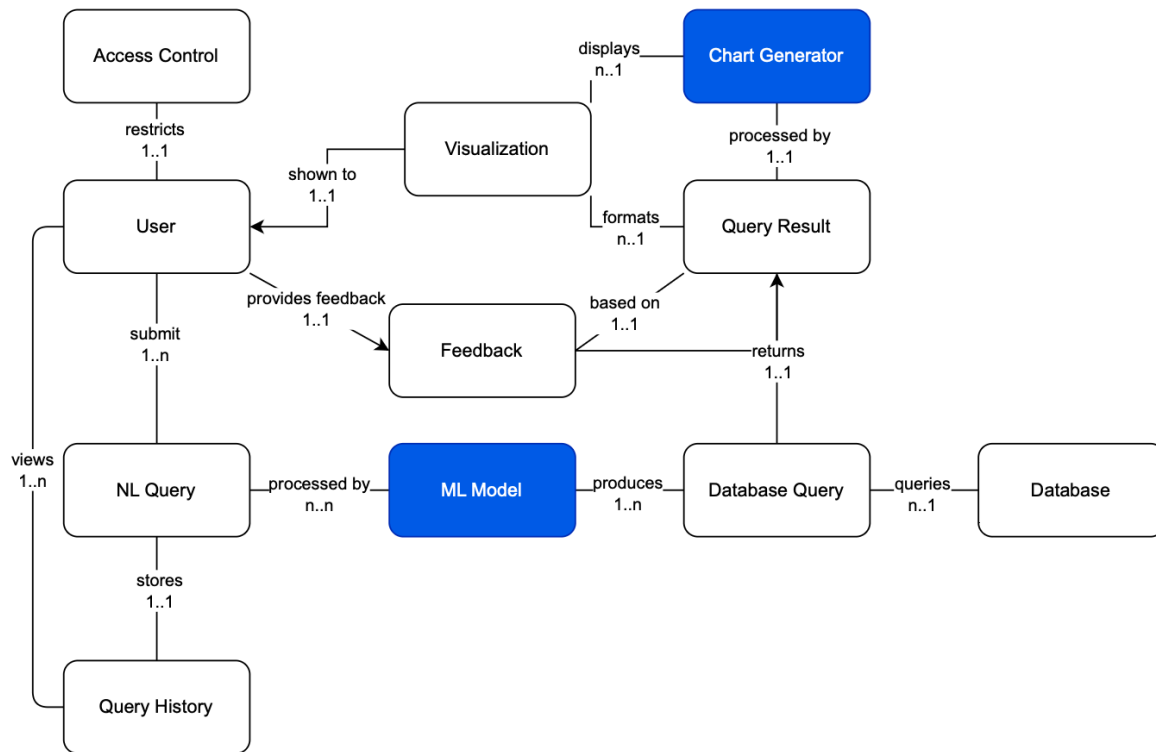
Implemented Use Cases

- ☒ **UC01** - Natural Language Translation to Domain-specific SQL
- ☒ **SUC01** - Process and validate Natural language
- ☐ **SUC01-2** - Record translation Request
- ☒ **UC02** - Execute valid query on database and display result
- ☒ **SUC02-01** - Format Result and Display it
- ☒ **UC03** - Display generated Query
- ☐ **SUC03-1** - Display debug logs
- ☐ **UC04** - Feedback from user
- ☐ **SUC04-1** - User feedback is recorded and stored
- ☒ **UC05** - User is able to configure database connection
- ☒ **SUC05-1** - Database credentials are validated

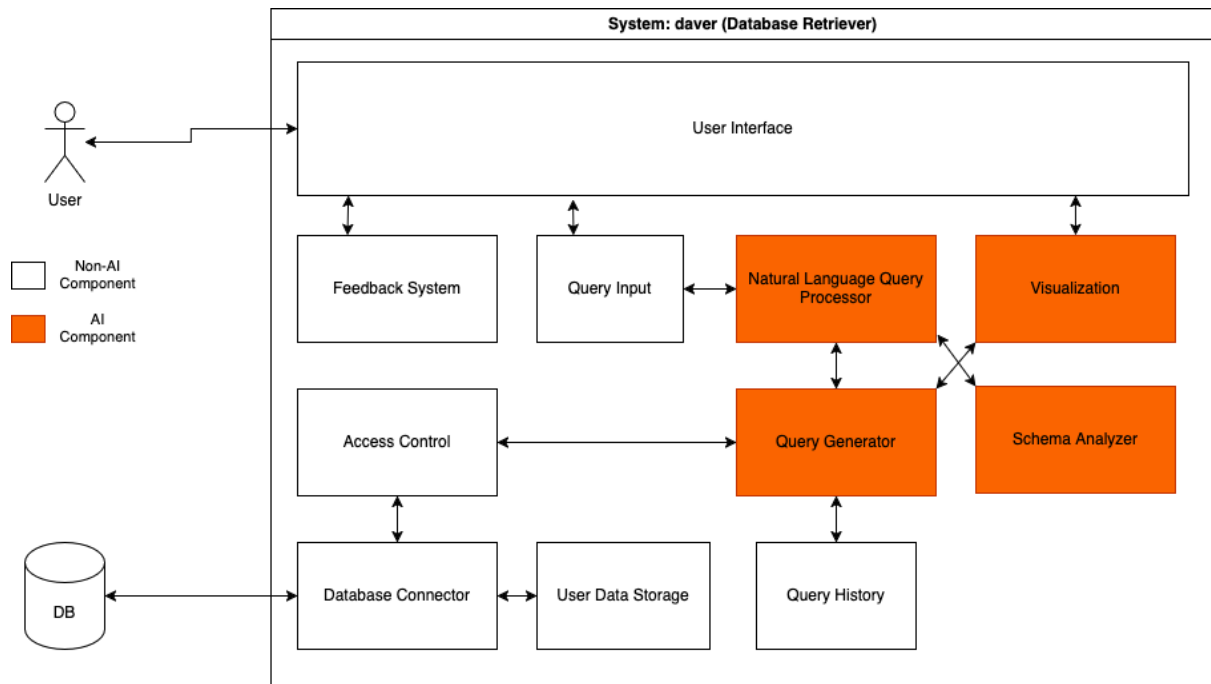
Traceability Matrix

Use cases	UC01	SUC01 -1	SUC01 -2	UC02	SUC02 -1	UC03	SUC03 -1	UC04	SUC04 -1	UC05	SUC05 -1
Requirements											
Req01	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
Req02	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
Req03	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
Req04	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
Req05	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
Req06	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
Req07	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
Req08	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
Req09	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
Req10	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
Req11	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
NfReq01	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
NfReq02	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
NfReq03	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
NfReq04	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE
NfReq05	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
NfReq06	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
NfReq07	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
NfReq08	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
NfReq09	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
NfReq10	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE

Domain Model



Architecture Diagram



Components Description

Non-AI Components:

1. User Interface

- a. Provides an interactive user interface for users to input natural language queries and view results in various formats. Offers controls for query history, feedback, and visualization options.
- b. Related to Req02, Req03, Req04, Req05, NfReq03.

2. Query Input

- a. Accepts and validates natural language queries from users, providing auto-completion and suggestions based on history. Allows users to refine queries and add additional context interactively.
- b. Related to Req02, Req09, Req10.

3. Database Connector

- a. Provides standardized interfaces to connect to different database systems (MongoDB, PostgreSQL, MySQL). Manages connections, query execution, and result retrieval.
- b. Related to Req02, NfReq02, NfReq05.

4. Feedback System

- a. Captures user feedback on query results and translations for continuous system improvement. Provides mechanisms for users to indicate incorrect results and suggest better alternatives.
 - b. Related to Req05, Req10, NfReq07.
- 5. User Data Storage**
 - a. Securely stores user profiles, preferences, and authentication information. Manages user-specific settings without storing sensitive database query results.
 - b. Related to Req11, NfReq04, NfReq08.
- 6. Query History**
 - a. Maintains a record of executed queries for reference, reuse, and improvement of the system. Stores query patterns and user interactions to enhance future translations.
 - b. Related to Req11, NfReq07.
- 7. Security & Access Control**
 - a. Enforces access restrictions based on user permissions and database security policies. Prevents unauthorized operations and ensures compliance with data privacy requirements.
 - b. Related to Req07, Req08, NfReq04, NfReq11.

AI Components:

- 8. Natural Language Query Processor**
 - a. Analyzes and understands the natural language queries using NLP techniques. Determines query intent, extracts entities, and identifies relationship requirements in user queries.
 - b. Related to Req01, Req09, NfReq03, NfReq06, NfReq09.
- 9. Query Generator**
 - a. Transforms the processed natural language query into optimized database-specific queries (SQL, MongoDB queries, etc.). Generates appropriate code with proper syntax for the target database system.
 - b. Related to Req01, Req03, Req06, NfReq01, NfReq06.
- 10. Schema Analyzer**
 - a. Automatically analyzes and understands database schemas, relationships, and semantics. Maintains models of database structures to improve query accuracy and adaptation to schema changes.
 - b. Related to Req06, Req07, Req08.
- 11. Visualization**
 - a. Transforms query results into appropriate visual representations (charts, graphs, tables) based on data type and context. Provides interactive visualization options for data exploration.
 - b. Related to Req04.

Design Questions

How should the user interface be structured ?

The user interface for daver should emulate a sort of chat window, similar to other well known LLM applications, with embedded widgets for visualizations, data tables etc.

How can we ensure natural language processing handles both technical and non-technical expressions effectively for database queries?

We will combine 3 Systems, one for translating what the user wrote into an instruction and a more concise, technical instruction set for generating the query. Another system will establish a sort of knowledge base about the database schema, data types as well as attributes of columns and tables, which will be referenced by the first system. The third will use said technical query and create the actual domain specific query in the correct language and syntax.

What strategy should we implement to maintain acceptable performance when dealing with complex joins or aggregations in translated queries?

We will use windowing or pagination of query data, as well as an estimation for the time cost of queries the full data, that can be presented to the user, who can then trigger that full query knowing its computational costs.

How should the system handle ambiguous natural language queries that could have multiple valid database interpretations?

The system will have a past-experience based scoring system that picks the most likely candidate and presents it. When user feedback indicates this to be wrong, the alternatives will be presented for the user to choose and this feedback will be incorporated into the parameters of the scoring system.

How will the system adapt to changes in database schemas?

The system will maintain and check the state of the schema, and when differences are detected, re-build its schema knowledge base.

What mechanisms should be implemented to prevent potentially harmful or resource-intensive queries?

We'll implement a domain specific security strategy for preventing the system from accessing certain tables or columns, as well as disabling certain query commands, such as allowing just read-only commands. Furthermore as part of the user credentials process a database admin can define forbidden tables, rows or columns, which are not allowed to be queried.

What visualization techniques would be most appropriate for different types of query results?

We'll create a decision tree algorithm that selects visualizations based on: data type, relationship types, cardinality, and user preferences. There will be standard data tables and pre-fabricated charts, but there is also a system for an LLM to dynamically create charting code that fits the data.

How can we effectively collect and incorporate user feedback to improve future query translations?

We'll implement a continuous learning system where user corrections directly influence future query translations. Explicit feedback (thumbs up/down, corrections) will have immediate impact, while implicit feedback (query refinements, result usage patterns) will influence longer-term model updates or changes to the knowledge base, as well as affecting the scoring system.

How should support for multiple databases be implemented?

We'll maintain separate modules for each supported database system (MongoDB, PostgreSQL, MySQL) that check for validity and syntax of queries, and can provide feedback to the query generation model, to adapt erroneous queries.

How should the system manage database connection security while still allowing natural language access?

We'll implement a credential isolation pattern where the application maintains database connections through service accounts with carefully scoped permissions. User permissions will be mapped to database role capabilities without exposing actual credentials. All database operations will be logged with both originating user and executing service information for audit purposes.